

Speaker Notes

Deep Learning Fundamentals · Batch 6

Naskah lisan per slide + panduan membaca rumus

Navasena AI

7 September 2026

Catatan pengajar untuk mendampingi deck `module02_slides.pdf` (72 slide, hari 2). Tiap entri diberi label **Slide N/72** sesuai nomor halaman deck, berisi naskah yang bisa langsung diucapkan (**Ucapkan**) dan, kalau slide-nya memuat rumus, cara membacanya dengan lantang (**Baca rumus**). Naskah ini bukan bacaan ulang teks slide: slide memuat yang perlu terlihat, notes menyambungkan idenya dan menyebut batas berlakunya. Frame bertanda *Intuisi* enak dibawa santai dengan tempo normal; frame mekanisme (rumus, TikZ berlapis, tabel parameter) dibaca lebih pelan dan sebaiknya berhenti sebentar setelah tiap butir. Sapaan yang dipakai: “kamu” saat mengajak mencoba, “kita” saat menalar bersama. Setiap analogi di deck ini punya pasangan mekanismenya di frame berikutnya, jadi jangan berhenti di analogi: turun bukit kembali ke gradient descent, bisik berantai kembali ke vanishing gradient, meminjam mata terlatih kembali ke backbone yang dibekukan, dan dua pemain kembali ke generator melawan discriminator. Angka di naskah ini diambil dari gambar dan tabel yang benar-benar tampil di slide; kalau figurnya diregenerasi dan angkanya berubah, angka di naskah ikut disesuaikan. Satu figur, `gpu_benchmark.pdf` di Slide 56, masih placeholder dan diisi dari run nb05 di Colab T4 pada hari pelaksanaan.

Glosarium: cara menyebut simbol dan singkatan

Pakai pembacaan ini konsisten sepanjang hari, termasuk saat menjawab pertanyaan.

Simbol / singkatan	Dibaca (lantang)
x, x_1, x_2, x_3	“x”, “x satu, x dua, x tiga” (nilai masukan satu neuron)
w, w_1, w_2, w_3	“w”, “w satu, w dua, w tiga” (bobot)
W, W_1, W_2	“W besar”, “W satu, W dua” (matriks bobot satu layer)
W_x, W_h	“W x” (bobot untuk masukan), “W h” (bobot untuk hidden state)
b	“b” (bias); b_1, b_2 = “b satu, b dua”
$w \cdot x$	“w titik x” (jumlah perkalian bobot dengan masukan)
$\hat{y}$	“y topi” (prediksi model)
σ	“sigma”; $\sigma(z)$ dibaca “sigmoid dari z”

Simbol / singkatan	Dibaca (lantang)
Σ	“sigma besar” (tanda penjumlahan di gambar neuron)
$\sum_{i,j}$	“sigma, untuk semua i dan j” (jumlahkan seluruh posisi filter)
η	“eta”, yaitu learning rate
$\partial L / \partial w$	“turunan loss terhadap w” (kemiringan loss di arah w)
$\leftarrow$	“menjadi” atau “diganti dengan”
h_t, h_{t-1}	“h ke-t” (hidden state langkah ini), “h ke-t minus satu” (langkah sebelumnya)
x_t	“x ke-t” (masukan pada langkah ke-t)
$\tanh$	dibaca “tan h”
$\log$	“log” ; $-\log p$ dibaca “minus log p”
$\max(0, x)$	“maksimum dari nol dan x”
ε	“epsilon” (angka kecil, mis. penstabil pembagian)
$f(w)$	“f dari w” (nilai loss pada bobot w)
$\times$	“kali” ; 784×128 dibaca “784 kali 128”
$\rightarrow$	“menjadi”
FP16, float16	dieja: “F P enam belas” / “float enam belas”
FP32, float32	dieja: “F P tiga dua” / “float tiga dua”
MAE	dieja: “M A E” (mean absolute error)
CNN, RNN, GRU	dieja: “C N N”, “R N N”, “G R U”
LSTM	dieja: “L S T M”
GAN	dibaca “gan” (satu suku kata) atau dieja “G A N”, pilih satu dan konsisten
GPU, CPU	dieja: “G P U”, “C P U”
ONNX	dibaca “onniks”
TensorRT	dibaca “Tensor R T”
VAE	dieja: “V A E”
ReLU	dibaca “relu”

*Tip pengucapan: bilangan desimal dibaca dengan “koma”, mis. $0,88 =$ “nol koma delapan delapan”; pangkat dibaca “dipangkatkan” atau “dikuadratkan”; nama layer dan metode di kode disebut apa adanya, mis. *Dense* dibaca “dense”, *fit()* dibaca “fit”.*

Daftar Isi

1	Pembuka	5
	Slide 1: Judul	5
	Slide 2: Peta hari 2	5
	Slide 3: Dari hari 1 ke hari 2	5
2	Act 1 — Bagaimana neural network belajar?	5
	Slide 4: Act 1 (pembuka)	5
	Slide 5: Intuisi: Satu neuron	6
	Slide 6: Dari satu neuron ke satu arsitektur	6
	Slide 7: Yang berubah saat model belajar	6
	Slide 8: Loss	6
	Slide 9: Intuisi: Gradient descent	7
	Slide 10: Learning rate	7
	Slide 11: Intuisi: Backprop	8
	Slide 12: Epoch dan batch	8
	Slide 13: Kurva train dan validation	8
	Slide 14: Overfitting, dropout, early stopping	8
	Slide 15: Normalisasi input	9
	Slide 16: Evaluasi multi-kelas	9
	Slide 17: Ringkasan Act 1	9
3	Act 2 — Kenapa butuh aktivasi?	10
	Slide 18: Act 2 (pembuka)	10
	Slide 19: Tanpa aktivasi tetap linear	10
	Slide 20: Empat aktivasi dan turunannya	10
	Slide 21: Intuisi: Vanishing gradient	10
	Slide 22: ReLU dan Leaky ReLU	11
	Slide 23: Softmax	11
	Slide 24: Intuisi: Optimizer	11
	Slide 25: Kapan mengganti default	12
	Slide 26: Ringkasan Act 2	12
4	Act 3 — Bagaimana komputer melihat gambar?	12
	Slide 27: Act 3 (pembuka)	12
	Slide 28: Gambar sebagai tabel angka	13
	Slide 29: Intuisi: Konvolusi	13
	Slide 30: Mekanisme konvolusi	13
	Slide 31: Feature map	13
	Slide 32: Pooling	14
	Slide 33: Arsitektur CNN nb03	14
	Slide 34: Augmentasi sebagai layer	14
	Slide 35: Intuisi: Transfer learning	15

Slide 36: Bekukan backbone, latih kepala	15
Slide 37: Kapan transfer learning menolong	15
Slide 38: Ringkasan Act 3	16
5 Act 4 — Bagaimana model membaca urutan?	16
Slide 39: Act 4 (pembuka)	16
Slide 40: Data berurutan	16
Slide 41: Intuisi: RNN	16
Slide 42: Mekanisme RNN	17
Slide 43: Intuisi: Urutan panjang	17
Slide 44: LSTM	17
Slide 45: GRU	18
Slide 46: Embedding	18
Slide 47: Hasil nyata pada data sinus	18
Slide 48: Kapan model urutan cocok	19
Slide 49: Ringkasan Act 4	19
6 Act 5 — Kapan GPU benar-benar diperlukan?	19
Slide 50: Act 5 (pembuka)	19
Slide 51: Training dan perkalian matriks	19
Slide 52: Intuisi: CPU vs GPU	20
Slide 53: Kapan GPU tidak untung	20
Slide 54: Mixed precision	20
Slide 55: ONNX dan TensorRT	21
Slide 56: Benchmark GPU	21
Slide 57: Kapan GPU sepadan	21
Slide 58: Ringkasan Act 5	22
7 Act 6 — Bagaimana model menghasilkan data baru?	22
Slide 59: Act 6 (pembuka)	22
Slide 60: Diskriminatif vs generatif	22
Slide 61: Intuisi: Autoencoder	22
Slide 62: Latent space dan interpolasi	23
Slide 63: Intuisi: GAN	23
Slide 64: Hasil GAN	23
Slide 65: Diffusion	24
Slide 66: Peta keluarga generatif	24
Slide 67: Kapan model generatif cocok	24
Slide 68: Ringkasan Act 6	24
8 Penutup	25
Slide 69: Enam notebook	25
Slide 70: Apa selanjutnya	25
Slide 71: Latihan mandiri	25
Slide 72: Terima kasih	26

Pembuka

Slide 1/72 Judul: Deep Learning Fundamentals

Ucapkan: "Selamat pagi, dan selamat datang di hari kedua. Hari ini Deep Learning Fundamentals: enam notebook, TensorFlow dan Keras, semuanya di Google Colab. Kemarin kita bekerja dengan model yang koefisiennya bisa dibaca satu per satu; hari ini modelnya punya ratusan ribu parameter, dan yang perlu kita pahami adalah bagaimana angka sebanyak itu bisa diperbaiki sendiri."

Slide 2/72 Peta hari 2

Ucapkan: "Seluruh isi hari ini dalam satu halaman: enam kartu, enam notebook. nb01 membangun neural network pertama di Fashion-MNIST. nb02 membandingkan fungsi aktivasi dan optimizer di data yang sama. nb03 pindah ke gambar berwarna, CIFAR-10, dengan CNN dan transfer learning. nb04 masuk ke data berurutan: RNN dan LSTM. nb05 soal GPU: mixed precision, ONNX, TensorRT. nb06 model generatif: autoencoder, GAN, diffusion. Yang perlu dicatat dari peta ini adalah pembagian waktunya. Dua blok pertama, nb01 dan nb02, isinya cara satu neural network belajar, dan itu bagian yang dipakai ulang di empat notebook sisanya. Kalau dua blok pertama terasa lambat, itu memang disengaja."

Slide 3/72 Dari hari 1 ke hari 2

Ucapkan: "Sebelum masuk, satu halaman untuk menyambung dari kemarin. Di kolom kiri, ML klasik: featurenya kita pilih dan rancang sendiri, kolom umur, gaji, durasi panggilan; datanya tabular, satu baris satu contoh; modelnya kecil dan koefisiennya bisa dibaca. Di kolom kanan, deep learning: featurenya dipelajari dari data mentah, pixel dan urutan kata; datanya gambar, teks, dan sinyal berurutan; modelnya berlapis dengan parameter ratusan ribu ke atas. Kotak di bawah itu yang paling penting hari ini. Alur kerjanya tidak berubah: data dipisah menjadi train, validation, dan test; setiap model dibandingkan dengan baseline; pilihan model diputuskan di data validation dan data test dibuka sekali di akhir. Yang berganti hanya bentuk modelnya dan dari mana featurenya datang. Jadi disiplin yang kamu pakai kemarin tetap berlaku, bukan diganti."

Act 1 — Bagaimana neural network belajar?

Slide 4/72 Act 1: Bagaimana neural network belajar?

Ucapkan: "Blok pertama, dan yang paling menentukan sisa hari ini. Pertanyaannya: apa sebenarnya yang terjadi di dalam `fit()` pada satu neural network. Jawabannya satu angka kesalahan, lalu diperbaiki sedikit demi sedikit."

Slide 5/72 Intuisi: Satu neuron: jumlah berbobot, lalu diputuskan

Ucapkan: "Mulai dari satu neuron, karena seluruh model hari ini hanya tumpukan dari benda ini. Tiga masukan di kiri, masing-masing punya satu bobot. Bobot itu takaran, seperti resep: kalau satu bahan porsi dinaikkan, pengaruhnya ke hasil akhir ikut naik. Ketiga masukan dikalikan bobotnya lalu dijumlahkan di bulatan sigma besar itu, ditambah bias, dan hasilnya lewat satu fungsi aktivasi sebelum menjadi keluaran. Bias gunanya menggeser hasilnya: dengan bias, neuron bisa aktif walau semua masukannya kecil. Analoginya berhenti di sini; mekanismenya cuma dua operasi, satu jumlah berbobot dan satu fungsi. Yang berubah saat model belajar adalah bobot dan bias itu, bukan bentuk rumusnya."

Baca rumus: tulisan kecil di bawah bulatan sigma, $w_1x_1 + w_2x_2 + w_3x_3 + b$, dibaca "w satu kali x satu, tambah w dua kali x dua, tambah w tiga kali x tiga, tambah b". Bentuk ringkasnya, $y = \sigma(w \cdot x + b)$, dibaca "y sama dengan sigma dari w titik x tambah b": "w titik x" berarti tiap bobot dikalikan masukannya lalu semuanya dijumlahkan.

Slide 6/72 Dari satu neuron ke satu arsitektur

Ucapkan: "Sekarang neuron tadi disusun berlapis, dan ini persis model nb01. Di kiri, 784 angka. Angka itu datang dari gambar 28 kali 28 yang digelar menjadi satu baris oleh layer **Flatten**. Lalu 128 neuron, masing-masing menghitung jumlah berbobot dari semua 784 angka itu, lewat ReLU. Lalu 64 neuron yang bekerja di atas keluaran 128 neuron sebelumnya, bukan lagi di atas pixel. Terakhir 10 neuron, satu untuk tiap kategori pakaian, dengan softmax. Garis-garis di gambar menunjukkan tiap neuron terhubung ke semua neuron layer sebelumnya, dan itu arti kata Dense. Yang berguna dipahami: layer kedua menyusun feature dari feature. Itu sebabnya modelnya disebut berlapis, dan itu sebabnya featurenya tidak perlu kita rancang sendiri seperti kemarin."

Slide 7/72 Yang berubah saat model belajar

Ucapkan: "Model yang sama, sekarang dihitung parameternya. Layer Dense 128: bobotnya 784 kali 128, ditambah 128 bias, jadi 100.480 angka. Layer Dense 64: 128 kali 64 ditambah 64 bias, 8.256 angka. Output layer: 64 kali 10 ditambah 10 bias, 650 angka. Totalnya 109.386. Seratus sembilan ribu angka itu dimulai dari nilai acak, dan seluruh proses training adalah memperbaiki angka-angka itu. Sekarang bedakan dengan baris kedua di bawah. Jumlah layer, jumlah neuron per layer, dan learning rate kamu tentukan sebelum training dimulai, dan ketiganya tidak ikut diperbaiki oleh training. Itu hyperparameter, dan cara memilihnya sama seperti kemarin: dibandingkan di data validation. Tanya ke kelas: dari 109.386 angka ini, berapa yang kita tulis sendiri? Nol."

Slide 8/72 Loss: satu angka yang mengukur seberapa salah

Ucapkan: "Supaya seratus ribu angka itu bisa diperbaiki, kesalahan model harus diringkas jadi satu angka. Itu loss. Untuk klasifikasi, modelnya mengeluarkan satu probabilitas untuk tiap kelas, dan loss hanya melihat satu di antaranya: probabilitas yang diberikan ke kelas yang benar. Kalau

model memberi 0,85 untuk kelas yang benar, lossnya kecil. Kalau memberi 0,05, lossnya besar. Loss satu batch adalah rata-rata loss tiap contohnya, dan angka itulah yang diperkecil. Satu hal yang sering tertukar: accuracy dilaporkan supaya kita bisa membacanya, tetapi bukan accuracy yang dipakai model untuk memperbaiki diri. Accuracy melompat-lompat, loss berubah halus, dan yang bisa diturunkan sedikit demi sedikit hanya yang berubah halus. Namanya di Keras `sparse_categorical_crossentropy`.”

Baca rumus: $\text{loss satu contoh} = -\log(p_{\text{kelas benar}})$ dibaca “loss satu contoh sama dengan minus log dari p kelas benar”. Tambahkan penjelasan lisan: “kalau p mendekati satu, lognya mendekati nol, jadi lossnya kecil; kalau p mendekati nol, lognya makin negatif, dan tanda minus di depan membuat lossnya membesar”.

Slide 9/72 Intuisi: Gradient descent: menuruni bukit dalam kabut

Ucapkan: “Bayangkan kamu berdiri di lereng bukit dan kabutnya tebal. Bentuk seluruh bukit tidak terlihat. Yang masih bisa kamu rasakan hanya kemiringan tanah di bawah kaki, jadi satu langkah diambil ke arah yang menurun, lalu dirasakan lagi, lalu melangkah lagi. Itu gradient descent, dan mekanismenya sama persis: bukitnya adalah loss sebagai fungsi bobot, kemiringan di titik sekarang adalah turunan loss terhadap bobot, dan satu langkah berarti bobotnya digeser sedikit ke arah yang menurunkan loss. Ketiga panel di gambar memakai fungsi yang sama, $f(w) = (w - 3)^2 + 1$, dengan dasar di w sama dengan tiga. Yang berbeda hanya ukuran langkahnya. Titik-titik bernomor itu posisi bobot setelah tiap langkah, jadi kamu bisa melihat langkahnya menumpuk di panel kiri dan berhamburan di panel kanan.”

Slide 10/72 Learning rate: ukuran langkah dan gejalanya

Ucapkan: “Ukuran langkah tadi punya nama: learning rate. Kolom kiri, terlalu kecil. Lossnya memang turun, tetapi lambat; di panel kiri gambar sebelumnya, learning rate 0,05 memakai 12 langkah dan belum sampai ke dasar. Gejalanya di notebook: epoch habis sementara lossnya masih jauh dari datar. Kolom kanan, terlalu besar. Langkahnya melewati dasar lalu mendarat lebih tinggi dari sebelumnya, dan tiap langkah berikutnya makin jauh; itu panel kanan dengan learning rate 1,05. Gejalanya loss naik-turun tanpa arah, atau muncul NaN. Panel tengah, 0,5, sampai ke dasar dalam delapan langkah. Kotak jebakan di bawah: Adam memakai learning rate 0,001 sebagai default, dan itu titik awal yang wajar, bukan angka yang cocok untuk semua data. Kalau loss diam saja atau meledak sejak epoch pertama, yang pertama kamu ubah learning ratenya, bukan jumlah layernya.”

Baca rumus: aturan yang mendasari kedua kolom ini, walau tidak tercetak di slide, adalah $w \leftarrow w - \eta \cdot \partial L / \partial w$. Bacanya: “ w baru sama dengan w lama dikurangi eta kali turunan loss terhadap w ”. Sebut sekali bahwa “eta” itulah learning rate, dan tanda minus di depan yang membuat langkahnya mengarah turun, bukan naik.

Slide 11/72 Intuisi: Backprop: kesalahan dibagi ke belakang

Ucapkan: "Satu pertanyaan yang belum terjawab: turunan loss terhadap seratus ribu bobot itu dihitung bagaimana. Ikuti dua panah di gambar. Panah hijau di atas adalah arah maju: data masuk, tiap layer menghitung keluarannya, sampai muncul prediksi dan satu angka loss di ujung kanan. Panah oranye di bawah arah mundur. Dari loss, tiap layer menerima informasi seberapa besar sumbangannya terhadap kesalahan itu, dan meneruskan bagian yang relevan ke layer sebelumnya. Hasil akhirnya: tiap bobot mendapat satu angka, sumbangannya sendiri terhadap kesalahan. Bobot yang menyumbang lebih banyak digeser lebih jauh. Itu backpropagation, dan di Keras seluruhnya terjadi di dalam `fit()`. Yang perlu kamu ingat bukan turunan berantainya, tetapi urutannya: maju dulu untuk tahu salahnya berapa, baru mundur untuk tahu siapa yang salah."

Slide 12/72 Epoch, batch, dan satu langkah perbaikan

Ucapkan: "Tiga istilah yang akan terus muncul di log training, dan sering tertukar. Batch: sekelompok contoh yang dihitung bersama, di nb01 isinya 32 gambar. Satu batch memberi satu langkah perbaikan bobot, jadi tiap kotak di gambar punya panah ke bawah bertuliskan bobot diperbarui. Epoch: seluruh data train dilewati sekali. Di nb01 data trainnya 55.000 gambar; dibagi 32, jadi satu epoch berisi 1.718 langkah perbaikan. Jadi setelah delapan epoch, bobotnya sudah diperbarui belasan ribu kali. Ukuran batch punya kompromi: batch besar membuat arah langkahnya lebih tenang karena dihitung dari lebih banyak contoh, tetapi butuh memori lebih besar dan langkahnya lebih sedikit per epoch. Angka di slide dari nb01: 55.000 train, 5.000 validation, 10.000 test."

Slide 13/72 Kurva train dan validation dibaca berdampingan

Ucapkan: "Ini hasil training nb01 yang sebenarnya, delapan epoch, dan cara membacanya perlu kita sepakati sekarang karena gambar seperti ini muncul di lima notebook berikutnya. Dua panel. Panel kiri loss, panel kanan accuracy. Di tiap panel ada dua kurva: data train dan data validation. Di run ini kedua kurva loss masih turun bersama, dan akurasi validation berhenti di sekitar 0,88. Aturan bacanya: selama kedua kurva loss masih turun bersama, model masih belajar hal yang berlaku umum. Kalau loss validation mulai naik sementara loss train terus turun, saat itulah model mulai menghafal data train. Satu batas dari run ini: garis putus-putus menandai epoch dengan val_loss terendah, dan pada run ini titik itu jatuh di epoch terakhir. Artinya delapan epoch belum cukup untuk melihat titik baliknya, jadi jangan simpulkan modelnya sudah selesai belajar."

Slide 14/72 Overfitting: melebarnya jarak train dan validation

Ucapkan: "Supaya titik baliknya terlihat, gambar ini dibuat sengaja: model yang sama dilatih pada 6.000 sampel train saja, jadi ada lebih sedikit contoh untuk dihafal ulang. Lihat selisih kedua panel. Tanpa dropout, jarak antara akurasi train dan akurasi validation melebar sampai 0,56. Dengan `Dropout(0,5)`, jaraknya 0,18. Jarak yang melebar itulah gejala overfitting, dan yang perlu dibaca adalah jaraknya, bukan angka train yang tinggi. Dua cara menahannya. Dropout mematikan sebagian neuron secara acak di setiap batch, jadi model tidak bisa bergantung pada

satu jalur saja. Early stopping berhenti melatih saat val_loss tidak lagi turun. Kotak jebakan: dropout hanya aktif saat training. Jadi kalau loss validation kamu lebih rendah daripada loss train, itu bukan keajaiban; saat evaluasi dropoutnya dimatikan sehingga seluruh neuron ikut bekerja.”

Slide 15/72 Normalisasi input: dari 0–255 ke 0,0–1,0

Ucapkan: ”Satu baris kode yang kelihatan sepele tetapi sering menentukan berhasil atau tidaknya training. Bar atas rentang pixel aslinya, nol sampai 255. Bar bawah setelah dibagi 255, jadi nol koma nol sampai satu koma nol. Alasannya kembali ke gradient descent. Masukkan dengan rentang besar menghasilkan gradient yang besar juga, sehingga satu learning rate sulit sekaligus pas untuk semua bobot: langkah yang aman untuk satu arah menjadi terlalu jauh untuk arah lain. Untuk pixel pembagiannya cukup konstanta 255, karena rentangnya sudah diketahui. Untuk data tabular caranya seperti kemarin: statistik scalingnya dihitung dari data train saja, lalu dipakai apa adanya ke validation dan test. Kalau statistiknya dihitung dari seluruh data sebelum split, informasi dari data test sudah bocor ke proses training.”

Slide 16/72 Evaluasi multi-kelas: akurasi saja belum cukup

Ucapkan: ”Angka akhir nb01: akurasi test 87,4 persen pada model Dense(128) di gambar ini, satu run, seed 42, dan data test disentuh sekali di akhir. Sekarang lihat matriksnya, karena satu angka 87,4 persen menyembunyikan bentuk kesalahannya. Baris menunjukkan kelas yang sebenarnya, kolom menunjukkan kelas yang diprediksi. Diagonal berisi prediksi yang benar. Yang menarik justru sel di luar diagonal: di sana terlihat kelas apa tertukar dengan apa. Pasangan yang paling sering tertukar dihitung langsung dari matriksnya di nb01, dan pasangan itu bisa berbeda tiap kali model dilatih ulang, jadi jangan dihafal dari slide. Polanya masuk akal: kelas dengan bentuk khas lebih jarang tertukar daripada kelas yang potongannya mirip. Tanya ke kelas: kalau modelnya dipakai untuk memilah pakaian di gudang, dua kelas mana yang paling merugikan kalau tertukar? Jawabannya menentukan metrik mana yang perlu kamu laporkan selain akurasi.”

Slide 17/72 Ringkasan Act 1

Ucapkan: ”Lima hal dari blok ini, dan kelimanya dipakai sampai malam. Satu, neuron menghitung jumlah berbobot dari masukannya, menambah bias, lalu melewatkannya ke aktivasi; layer adalah kumpulan neuron seperti itu. Dua, loss meringkas kesalahan menjadi satu angka, gradient descent menurunkannya dengan langkah kecil ke arah menurun, dan backprop yang membagi kesalahan itu ke tiap bobot. Tiga, learning rate menentukan ukuran langkah: terlalu kecil membuat training lambat, terlalu besar membuatnya meledak, dan itu hal pertama yang diperiksa saat loss tidak turun. Empat, kurva train dan validation dibaca bersama; jarak yang melebar tanda overfitting, dan dropout serta early stopping menahannya. Lima, untuk klasifikasi banyak kelas, confusion matrix menunjukkan kesalahan yang tidak terlihat dari angka akurasi.”

Act 2 — Kenapa butuh aktivasi?

Slide 18/72 Act 2: Kenapa butuh aktivasi?

Ucapkan: "Blok kedua, dan pendek. Di Act 1 kata aktivasi muncul tiga kali tanpa dijelaskan kenapa harus ada. Pertanyaannya: apa yang hilang kalau aktivasinya dihapus. Jawaban singkatnya, tanpa aktivasi seratus layer sama saja dengan satu layer."

Slide 19/72 Tanpa aktivasi, tumpukan layer tetap satu garis

Ucapkan: "Tiga layer ditumpuk di kiri, masing-masing punya matriks bobot dan bias. Tanda sama dengan di tengah itu bukan kiasan: tumpukan itu benar-benar bisa ditulis ulang sebagai satu layer, yang di kanan. Buktinya satu baris di bawah gambar. Konsekuensinya keras: kalau semua layer hanya menghitung jumlah berbobot tanpa aktivasi, menambah layer tidak menambah satu pun bentuk yang bisa dipelajari model. Model dengan sepuluh layer linear punya kemampuan yang sama dengan satu layer linear, hanya dengan lebih banyak parameter dan waktu training lebih lama. Karena itu nb02 memakai model tanpa aktivasi sebagai baseline linear lebih dulu, lalu membandingkannya dengan model beraktivasi di data yang sama. Selisihnya itu yang menjawab pertanyaan act ini dengan angka, bukan dengan klaim."

Baca rumus: $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$ dibaca "W dua, dikalikan tanda kurung W satu x tambah b satu, tambah b dua, sama dengan tanda kurung W dua W satu, dikalikan x, tambah tanda kurung W dua b satu tambah b dua". Lalu tunjuk kedua tanda kurung di kanan: "yang di depan x itu satu matriks bobot baru, yang di belakang satu bias baru, jadi bentuknya kembali menjadi satu layer".

Slide 20/72 Empat aktivasi dan turunannya

Ucapkan: "Empat aktivasi yang dipakai nb02, dan gambar ini punya dua baris yang perlu dibedakan. Baris atas fungsinya: bentuk keluaran neuron untuk tiap nilai masukan. Baris bawah turunannya, dan turunan inilah yang dipakai backprop untuk membagi kesalahan ke belakang. Jadi baris bawah yang menentukan bobotnya bergerak atau tidak. Sekarang lihat daerah merah di baris bawah. Di sana turunannya mendekati nol, artinya kalau masukan satu neuron jatuh di daerah itu, bobotnya hampir tidak berubah walau modelnya masih salah. Sigmoid dan tanh punya daerah merah di kedua ujungnya, dan itu yang jadi masalah di slide berikutnya. Bandingkan dengan ReLU: turunannya rata satu di seluruh sisi positif, tanpa daerah merah di sana."

Slide 21/72 Intuisi: Vanishing gradient: pesan yang habis di jalan

Ucapkan: "Pernah main bisik berantai? Satu kalimat dibisikkan dari orang pertama ke orang terakhir, dan yang keluar di ujung sering sudah tidak menyerupai kalimat aslinya. Sinyal perbaikan di neural network yang dalam mengalami hal serupa, dan mekanismenya bisa dihitung. Turunan sigmoid nilai terbesarnya 0,25. Saat kesalahan dibagi ke belakang, tiap kali melewati

satu layer sigmoid angkanya dikalikan paling banyak 0,25. Ikuti angka di gambar: mulai dari satu di loss, lalu dikali 0,25 tiga kali, dan yang sampai ke layer pertama sekitar 0,016. Jadi layer paling awal menerima sinyal perbaikan yang lebih kecil dari dua persen sinyal aslinya, dan hampir tidak ikut belajar. Bedanya dengan bisik berantai: di sini penyusutannya bukan karena salah dengar, tetapi perkalian angka yang bisa kita hitung. Itu namanya vanishing gradient.”

Slide 22/72 ReLU: turunannya tetap 1 di sisi positif

Ucapkan: ”Jawaban paling banyak dipakai untuk masalah tadi, dan bentuknya sesederhana dua garis di kiri. ReLU meneruskan nilai positif apa adanya dan menolak yang negatif. Karena sisi positifnya garis lurus dengan kemiringan satu, turunannya di sana tetap satu, jadi kesalahan tidak menyusut saat dibagi ke belakang melewatinya. Hitungannya juga murah: satu perbandingan dengan nol per neuron, tanpa fungsi eksponensial. Tetapi ada harganya, dan itu butir ketiga. Sisi negatifnya nol rata, jadi turunannya nol juga. Neuron yang masukannya negatif terus akan berhenti diperbarui dan tidak punya jalan kembali; keadaan itu disebut dead ReLU. Leaky ReLU, gambar bawah, memberi kemiringan kecil di sisi negatif, 0,1 pada gambar slide sebelumnya, sehingga masih ada gradient yang mengalir dan neuronnya bisa pulih.”

Baca rumus: $f(x) = \max(0, x)$ dibaca “f dari x sama dengan maksimum dari nol dan x”. Tambahkan sekali dengan kata biasa: “kalau x positif hasilnya x sendiri; kalau x negatif hasilnya nol”.

Slide 23/72 Softmax: skor mentah menjadi probabilitas

Ucapkan: ”Aktivasi yang tempatnya khusus: output layer klasifikasi banyak kelas. Bedanya dengan ReLU, softmax tidak bekerja per neuron, tetapi pada seluruh output layer sekaligus, karena keluarannya harus berjumlah satu. Ikuti angkanya di gambar. Skor mentah tiga kelas, 2,0 dan 1,0 dan 0,1, keluar menjadi 0,66 dan 0,24 dan 0,10, dan totalnya tepat satu. Yang perlu dilihat: selisih skor satu poin, dari 2,0 ke 1,0, menjadi selisih probabilitas yang cukup besar, 0,66 lawan 0,24. Penyebabnya skornya lewat fungsi eksponensial dulu, jadi selisih kecil di skor menjadi perbandingan yang tegas di probabilitas. Dua kasus lain: untuk dua kelas, satu keluaran dengan sigmoid sudah cukup; untuk regresi, output layer dibiarkan tanpa aktivasi karena keluarannya memang angka bebas.”

Baca rumus: bentuk yang dipakai softmax, $p_i = e^{z_i} / \sum_j e^{z_j}$, dibaca “p ke-i sama dengan e pangkat z ke-i, dibagi sigma untuk semua j dari e pangkat z ke-j”. Terjemahan lisannya: “tiap skor dipangkatkan e dulu, lalu dibagi jumlah seluruh hasil pangkat itu, sehingga totalnya menjadi satu”.

Slide 24/72 Intuisi: Optimizer: cara berbeda menuruni bukit yang sama

Ucapkan: ”Kembali ke bukit dari Act 1, sekarang dilihat dari atas. Garis oval itu seperti garis ketinggian di peta: satu oval berarti tempat dengan loss sama, dan titik putih di tengah adalah minimumnya. Tiga jalur, tiga optimizer, berangkat dari titik yang sama. Garis merah SGD:

melangkah menurut kemiringan saat itu saja, jadi jalurnya bolak-balik menyeberangi lembah. Garis oranye SGD dengan momentum: setiap langkah membawa sisa arah langkah sebelumnya, seperti bola yang sudah punya kecepatan, jadi belokannya tidak setajam itu. Garis hijau Adam: menyimpan rata-rata arah dan rata-rata besar gradient, lalu menyesuaikan ukuran langkah untuk tiap parameter secara terpisah. Yang perlu ditegaskan: ketiganya menuruni bukit yang sama. Optimizer mengubah cara melangkah, bukan bentuk bukitnya, jadi optimizer tidak memperbaiki data atau arsitektur yang salah.”

Slide 25/72 Kapan mengganti aktivasi atau optimizer default

Ucapkan: ”Halaman ini gunanya menahan kebiasaan mengganti-ganti aktivasi tanpa alasan. Kolom kiri, tanda yang wajar dijadikan alasan mengganti. Banyak neuron ReLU berhenti aktif dan keluarannya nol terus, itu dead ReLU dan Leaky ReLU pantas dicoba. Loss diam sejak epoch awal padahal datanya sudah dinormalisasi. Atau jaringannya dalam dan layer awalnya hampir tidak berubah, yang mengarah ke vanishing gradient. Kolom kanan, tiga keadaan yang sering disalahartikan. Lossnya belum turun karena learning ratenya yang belum disesuaikan; perbandingannya belum diuji di data validation, jadi belum ada dasarnya; atau modelnya masih underfit karena layernya memang terlalu sedikit. Kotak jebakan menyambung ke situ: dead ReLU biasanya berawal dari learning rate yang terlalu besar, bukan dari ReLU itu sendiri. Urutan yang hemat waktu: learning rate dulu, arsitektur kemudian.”

Slide 26/72 Ringkasan Act 2

Ucapkan: ”Lima hal dari blok aktivasi. Satu, aktivasi yang membuat tumpukan layer bisa mempelajari pola melengkung; tanpa aktivasi hasil akhirnya tetap satu layer linear. Dua, sigmoid dan tanh punya turunan yang mengecil di kedua ujung, sehingga kesalahan menyusut saat dibagi ke belakang melewati banyak layer. Tiga, ReLU menjaga turunannya tetap satu di sisi positif dan murah dihitung, jadi dipakai sebagai pilihan awal untuk hidden layer, sementara Leaky ReLU menyisakan gradient kecil di sisi negatif. Empat, softmax untuk output layer klasifikasi banyak kelas, sigmoid untuk dua kelas, tanpa aktivasi untuk regresi. Lima, optimizer hanya mengubah cara melangkah, bukan bukitnya; Adam titik awal yang wajar, dan learning ratenya tetap perlu diperiksa.”

Act 3 — Bagaimana komputer melihat gambar?

Slide 27/72 Act 3: Bagaimana komputer melihat gambar?

Ucapkan: ”Blok ketiga, dan mulai di sini modelnya punya bentuk khusus untuk jenis datanya. Pertanyaannya: apa yang perlu ditambahkan supaya model mengerti bahwa yang masuk itu gambar, bukan sekadar daftar angka. Jawabannya satu filter kecil yang sama, digeser ke seluruh gambar.”

Slide 28/72 Gambar sampai ke model sebagai tabel angka

Ucapkan: "Sebelum masuk ke konvolusi, kita lihat bentuk datanya. Kiri, potongan gambar abu-abu; tiap kotak satu pixel, dan nilainya nol sampai 255, nol paling gelap. Kotak hijau diperbesar di sebelahnya: empat angka, 235 dan 240 di atas, 220 dan 225 di bawah. Itu seluruh isi gambar bagi model, angka. Kanan, gambar berwarna: tiap pixel punya tiga nilai, merah, hijau, biru, jadi CIFAR-10 berukuran 32 kali 32 kali 3 sama dengan 3.072 angka per gambar, sementara satu gambar Fashion-MNIST 784 angka. Butir kedua yang menjadi alasan seluruh act ini. Dense layer membaca 3.072 angka itu sebagai satu daftar panjang: pixel yang bertetangga dan pixel di sudut yang berlawanan diperlakukan sama saja. Padahal pada gambar, kedekatan itu informasi."

Slide 29/72 Intuisi: Konvolusi: satu filter kecil digeser ke seluruh gambar

Ucapkan: "Bayangkan satu jendela kecil, tiga kali tiga, yang kamu geser ke seluruh permukaan gambar sambil bertanya satu hal yang sama di tiap posisi: di sini ada perubahan terang ke gelap ke arah samping atau tidak. Itu filter tepi vertikal, dan panel di gambar hasilnya: terang hanya di tempat yang punya perubahan ke arah samping, gelap di tempat yang rata. Mekanismenya persis itu, tanpa tambahan: satu filter, satu pertanyaan, digeser ke semua posisi, dan jawabannya berbentuk peta, di mana pola itu ditemukan. Satu catatan penting di keterangan kecil: kedua filter di gambar ini ditulis tangan supaya efeknya jelas. Di CNN sungguhan, sembilan angka di dalam filter itu tidak kita tulis; angkanya dipelajari dari data seperti bobot lainnya."

Slide 30/72 Mekanisme konvolusi: satu jumlah berbobot per posisi

Ucapkan: "Sekarang mekanismenya dalam angka. Input lima kali lima di kiri. Kotak hijau menandai posisi filter saat ini, tiga kali tiga, dan filternya di tengah: sembilan bobot dan satu bias. Di posisi itu, sembilan angka input dikalikan sembilan bobot filter lalu dijumlahkan, dan hasilnya satu angka, satu kotak hijau di peta keluaran sebelah kanan. Filter digeser satu langkah, hitung lagi, satu kotak lagi. Lima kali lima dengan filter tiga kali tiga memberi keluaran tiga kali tiga. Dua butir di bawah yang menjelaskan kenapa cara ini menang untuk gambar. Pertama, bobot filternya dibagi bersama: sembilan angka yang sama dipakai di seluruh gambar, jadi pola yang sama tetap dikenali walau posisinya berpindah. Kedua, hitung parameternya. `Conv2D(32, (3,3))` di nb03 memakai 896 parameter. Satu Dense layer dari 3.072 pixel ke 1.024 neuron butuh 3.145.728 bobot. Selisihnya bukan puluhan persen, tetapi ribuan kali."

Baca rumus: $\text{keluaran}(r, c) = \sum_{i,j} w_{ij} x_{r+i, c+j} + b$ dibaca "keluaran pada baris r kolom c sama dengan sigma, untuk semua i dan j, dari w ke-i-j dikalikan x pada baris r tambah i dan kolom c tambah j, lalu ditambah b". Sambil membaca, tunjuk kotak hijau di input dan kotak hijau di keluaran: "r dan c itu posisi jendelanya; i dan j berjalan di dalam jendela".

Slide 31/72 Feature map: satu peta untuk tiap filter

Ucapkan: "Satu filter memberi satu peta, dan satu layer konvolusi punya banyak filter. Delapan peta di gambar ini dari layer konvolusi pertama sebuah CNN kecil setelah satu epoch. Cara

membacanya: warna kuning berarti filter itu bereaksi kuat di posisi tersebut, gelap berarti hampir tidak bereaksi. Yang menarik, kedelapan filter itu belajar hal yang berbeda tanpa kita atur: ada yang menyorot goresan tebal, ada yang menyorot latar, dan ada yang masih hampir kosong karena satu epoch memang baru permulaan. Butir terakhir yang menyambung ke slide berikutnya: layer konvolusi kedua bekerja di atas peta ini, bukan di atas pixel mentah. Karena masukannya sudah berupa jawaban atas pertanyaan sederhana, pola yang dikenali layer berikutnya bisa lebih besar dan lebih abstrak.”

Slide 32/72 Pooling: menyusutkan peta, menyimpan yang paling kuat

Ucapkan: ”Operasi kedua di CNN, dan yang paling mudah dijelaskan. Feature map empat kali empat di kiri dibagi menjadi empat petak dua kali dua, garis hijau itu batasnya. Dari tiap petak diambil satu angka: yang terbesar. Petak kiri atas isinya satu, tiga, lima, enam, jadi keluarannya enam. Petak kanan atas memberi empat, kiri bawah empat, kanan bawah delapan. Empat kali empat menjadi dua kali dua, seperempat ukuran, dan tanpa satu pun bobot yang dilatih; `MaxPooling2D` tidak punya parameter. Di nb03 ini terjadi tiga kali: 32 menjadi 16, menjadi 8, menjadi 4. Dua akibatnya. Karena hanya nilai terkuat yang disimpan, pergeseran kecil pada gambar tidak banyak mengubah hasilnya. Dan karena petanya menyusut, layer setelahnya jauh lebih ringan dihitung.”

Slide 33/72 Arsitektur CNN nb03 dari ujung ke ujung

Ucapkan: ”Semua bagian tadi sekarang dirangkai, dan ini model nb03 apa adanya. Masuk 32 kali 32 kali 3. Blok pertama, `Conv2D 32` filter lalu `MaxPool`, keluar 16 kali 16 kali 32. Blok kedua, `Conv2D 64` lalu `MaxPool`, keluar 8 kali 8 kali 64. Blok ketiga, `Conv2D 128` lalu `MaxPool`, keluar 4 kali 4 kali 128. Baca angkanya berpasangan: ukuran petanya menyusut, jumlah filternya bertambah. Lalu `Flatten` menggelar 4 kali 4 kali 128 menjadi 2.048 angka, masuk `Dense 128` dengan `Dropout 0,5`, dan berakhir di `Dense 10` dengan `softmax`. Pembagian tugasnya jelas: bagian konvolusi menyusun feature dari pixel, bagian `Dense` di ujung memakai feature itu untuk memilih satu dari sepuluh kelas. Bagian ujung itu yang biasa disebut kepala klasifikasi, dan istilah itu dipakai lagi dua slide berikutnya.”

Slide 34/72 Augmentasi sebagai layer di dalam model

Ucapkan: ”Cara menambah ragam data train tanpa mengumpulkan gambar baru. Di kiri, tiga layer augmentasi dipasang di dalam model, tepat setelah `Input` dan sebelum `Conv2D`: `RandomFlip`, `RandomRotation(0,0.4)`, `RandomZoom(0,1)`. Karena bentuknya layer, keduanya ikut aturan Keras: aktif saat `fit()`, mati saat evaluasi. Jadi gambar yang sama muncul dibalik, diputar sedikit, atau di-zoom ulang di setiap epoch, dan model tidak melihat versi yang persis sama dua kali. Butir kedua yang perlu ditimbang tiap kali: ragamnya dipilih sesuai datanya. Kucing yang dibalik tetap kucing, tetapi angka atau tulisan yang dibalik bisa berubah arti. Kotak jebakan: karena gambar train diacak sementara data validation tidak, akurasi train bisa terlihat lebih ren-

dah daripada akurasi validation di awal training. Itu efek augmentasi, bukan tanda ada yang salah.”

Slide 35/72 Intuisi: Transfer learning: meminjam mata yang sudah terlatih

Ucapkan: ”Kalau kamu punya dua ribu gambar dan butuh model gambar yang bagus, melatih dari nol bukan satu-satunya jalan. Bayangkan meminjam mata orang yang sudah bertahun-tahun melihat gambar: dia sudah tahu apa itu tepi, apa itu tekstur, apa itu roda. Yang perlu diajarkan hanya nama-nama kelas di pekerjaanmu. Mekanismenya persis susunan kotak di gambar. ResNet50 sudah dilatih di ImageNet, 14 juta gambar dan 1.000 kelas. Layer awalnya mempelajari hal yang umum untuk gambar apa pun: tepi dan warna, lalu tekstur dan sudut, lalu bagian objek seperti roda atau mata. Bagian di dalam kotak putus-putus itu dipinjam apa adanya. Yang kita tambahkan hanya kotak hijau di kanan, kepala baru untuk sepuluh kelas CIFAR-10, dan hanya bagian itu yang dilatih dengan data kita.”

Slide 36/72 Mekanismenya: bekukan backbone, latih kepalanya

Ucapkan: ”Kalimat meminjam mata terlatih tadi, dalam kode, cuma satu baris: `base_model.trainable = False`. Susunannya di gambar: ResNet50 tanpa layer teratas, lalu `GlobalAveragePooling2D` yang meringkas peta menjadi satu vektor, lalu `Dense 128` dengan `Dropout`, lalu `Dense 10 softmax`. Dengan `trainable` bernilai `False`, backprop berhenti di batas kepala: hanya bobot kepala yang bergerak, dan bagian terbesar modelnya tetap seperti hasil latihannya di ImageNet. Satu harapan yang perlu diturunkan di sini. Waktu per epochnya tidak otomatis kecil, karena setiap gambar masih dihitung melewati seluruh ResNet50; yang lebih pendek hanya arah baliknya. Dan fine-tuning adalah langkah lanjutan, bukan langkah pertama: setelah kepalanya stabil, beberapa layer atas backbone dibuka lalu dilatih dengan learning rate kecil supaya bobot yang sudah baik tidak rusak.”

Slide 37/72 Kapan transfer learning menolong, kapan tidak

Ucapkan: ”Halaman yang paling sering dilewati padahal paling menghemat waktu. Kolom kiri, keadaan yang cocok: datanya sedikit, ribuan gambar bukan jutaan; resolusinya mendekati 224 kali 224, ukuran yang dipakai backbonenya belajar; dan objeknya sejenis foto sehari-hari yang ada di ImageNet. Kolom kanan, keadaan yang kurang cocok: gambarnya kecil, 32 kali 32 seperti CIFAR-10; domainnya jauh dari foto biasa, misalnya citra medis atau radar; atau datanya justru besar dan cukup untuk melatih CNN dari nol. Kotak jebakan menyebut hasilnya di nb03. ResNet50 yang dibekukan tidak otomatis mengalahkan CNN kecil yang dilatih dari nol pada CIFAR-10, karena gambar 32 kali 32 harus diperbesar dulu sebelum masuk backbone yang belajar di 224 kali 224, dan detail yang hilang tidak kembali. Jadi jangan disimpulkan dari nama modelnya; bandingkan keduanya di data validation.”

Slide 38/72 Ringkasan Act 3

Ucapkan: "Lima hal dari blok gambar. Satu, gambar masuk ke model sebagai tabel angka, dan konvolusi memanfaatkan kedekatan antar pixel yang diabaikan Dense layer. Dua, satu filter kecil digeser ke seluruh gambar dan menghasilkan satu feature map; bobot yang dibagi bersama membuat parameternya jauh lebih sedikit. Tiga, pooling menyusutkan peta dengan menyimpan nilai terkuat, tanpa menambah parameter. Empat, blok Conv lalu Pool ditumpuk sampai petanya kecil, lalu Flatten dan Dense mengambil keputusan kelas. Lima, augmentasi sebagai layer hanya aktif saat training, dan transfer learning meminjam backbone terlatih lalu melatih kepalanya, dengan keunggulan yang bergantung pada resolusi serta kedekatan domain."

Act 4 — Bagaimana model membaca urutan?

Slide 39/72 Act 4: Bagaimana model membaca urutan?

Ucapkan: "Blok keempat, dan bentuk data yang ketiga. Pertanyaannya: apa yang perlu ditambahkan supaya model tahu bahwa masukannya punya urutan. Jawabannya satu ringkasan pendek yang dibawa dari langkah ke langkah."

Slide 40/72 Data berurutan: posisinya ikut membawa arti

Ucapkan: "Tiga contoh data berurutan di satu halaman. Baris atas teks: Saya, suka, kucing. Tukar urutannya dan artinya berubah, jadi urutan itu sendiri adalah informasi. Baris tengah sensor suhu, satu nilai tiap menit selama berjam-jam. Baris bawah penjualan harian, dengan pola mingguan dan tren. Butir kedua yang menjelaskan kenapa dua act sebelumnya belum cukup: Dense layer dan CNN memproses tiap contoh tanpa membawa apa pun dari contoh sebelumnya. Untuk urutan, model perlu tempat menyimpan yang sudah dibaca. Dua data yang dipakai nb04: gelombang sinus sebagai contoh kecil yang bisa diperiksa dengan mata, lalu 50.000 review film IMDB yang dipotong menjadi 200 kata. Tanya ke kelas: data di pekerjaanmu, barisnya berurutan atau tidak? Kalau urutannya bisa diacak tanpa kehilangan arti, blok ini bukan yang kamu butuhkan."

Slide 41/72 Intuisi: RNN: satu catatan ringkas yang dibawa terus

Ucapkan: "Bayangkan kamu membaca satu review film sambil memegang satu kartu catatan kecil. Setiap kalimat yang lewat, catatan itu kamu perbarui, bukan kamu tambah; kartunya tetap satu dan ukurannya tetap. Di akhir review, keputusanmu diambil dari kartu itu. Itu RNN, dan kartu catatannya punya nama: hidden state, panah oranye yang melingkar di gambar. Mekanismenya: modelnya membaca satu langkah pada satu waktu, dan di setiap langkah catatan ringkasnya diperbarui dari catatan lama ditambah masukan baru. Batas yang perlu disebut supaya tidak keliru: yang tersimpan bukan seluruh riwayat, hanya ringkasan sepanjang beberapa puluh angka. Jadi RNN memang bisa lupa, dan itu bukan cacat implementasi, tetapi akibat dari ukuran

catatannya yang tetap.”

Slide 42/72 Mekanisme RNN: bobot yang sama dipakai di tiap langkah

Ucapkan: ”Gambar yang sama, sekarang dibentangkan supaya tiap langkah terlihat. Tiga kotak RNN di gambar itu bukan tiga sel berbeda; itu satu sel yang sama, digambar tiga kali untuk tiga langkah waktu. Masukan langkah pertama masuk, keluar h satu; h satu masuk ke langkah kedua bersama masukan kedua, keluar h dua; begitu seterusnya. Keluaran langkah terakhir, h tiga, yang masuk ke Dense untuk memberi prediksi. Kalimat kunci ada di bawah gambar: sel yang sama, bobot yang sama, dipakai ulang di setiap langkah. Jadi menambah panjang urutan tidak menambah parameter. Dua butir terakhir: matriks pertama membaca masukan langkah ini, matriks kedua membaca ringkasan langkah sebelumnya, dan keduanya tidak berubah sepanjang urutan. Panjang catatannya ditentukan jumlah unit: SimpleRNN(32) di nb04 membawa 32 angka antar langkah.”

Baca rumus: $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$ dibaca “ h ke- t sama dengan $\tanh$ dari W dikalikan x ke- t , tambah W dikalikan h ke- t minus satu, tambah b ”. Terjemahan lisannya: “catatan baru dihitung dari masukan langkah ini dan catatan langkah sebelumnya, lalu dilewatkan $\tanh$ supaya nilainya tetap di antara minus satu dan satu”.

Slide 43/72 Intuisi: Urutan panjang: pesan yang menipis di sepanjang rantai

Ucapkan: ”Bisik berantai kembali, kali ini rantainya bukan layer tetapi langkah waktu. Ikuti ketebalan panah oranye di bawah gambar, dari loss di kanan ke langkah pertama di kiri: makin ke kiri makin tipis. Mekanismenya sama seperti di Act 2, hanya tempatnya berpindah. Karena bobot yang sama dipakai ulang di tiap langkah, sinyal perbaikan dikalikan ulang setiap kali melewati satu langkah ke belakang, dan hasil perkalian berulang itu mengecil cepat. Sekarang pasang angkanya: review IMDB sepanjang 200 kata berarti 200 langkah. Jadi kata di awal review hampir tidak lagi mempengaruhi perbaikan bobot, padahal kalimat pembuka sebuah review sering justru yang paling menentukan penilaiannya. Masalah itu yang dijawab dua slide berikutnya.”

Slide 44/72 LSTM: tiga gerbang yang mengatur satu catatan

Ucapkan: ”Jawaban yang paling banyak dipakai, dan bentuknya jalur lurus di atas plus tiga gerbang di bawah. Garis hijau tebal di atas itu catatannya, cell state, dan yang perlu diperhatikan garisnya berjalan lurus dari langkah ke langkah. Tiga gerbang di bawah mengaturnya. Gerbang lupa memutuskan bagian mana dari catatan lama yang dibuang. Gerbang simpan memutuskan bagian mana dari masukan baru yang ditulis ke catatan. Gerbang keluar memutuskan bagian mana dari catatan yang dibaca sebagai keluaran langkah ini. Tiga hal yang perlu ditegaskan. Ketiga gerbang itu bukan aturan tetap yang kita tulis; bobotnya dilatih, jadi model belajar sendiri kapan sesuatu perlu dibuang, ditulis, atau dibaca. Jalur catatan itu tidak melewati perkalian bobot di tiap langkah, dan itulah sebabnya informasi dari awal urutan lebih tahan. Dan harganya nyata: satu sel LSTM menyimpan lebih banyak bobot dan lebih lambat dihitung daripada SimpleRNN

dengan jumlah unit yang sama.”

Slide 45/72 GRU: versi ringkas dengan dua gerbang

Ucapkan: ”Satu tabel untuk menempatkan ketiga sel berdampingan. SimpleRNN(32): tanpa gerbang, tanpa catatan terpisah, 1.088 parameter. GRU(32): dua gerbang, update dan reset, tanpa catatan terpisah, 3.360 parameter. LSTM(32): tiga gerbang, dengan catatan terpisah, 4.352 parameter. GRU menggabungkan gerbang lupa dan simpan menjadi satu gerbang update, dan tidak memisahkan catatan dari hidden state. Angka di tabel dihitung untuk 32 unit dan satu feature masukan, seperti pada data sinus nb04, jadi kalau jumlah featurenya berubah angkanya ikut berubah. Butir terakhir yang penting sebagai kebiasaan: pilihan antara GRU dan LSTM tidak diselesaikan dari jumlah gerbangnya, tetapi dengan mencoba keduanya dan membandingkan hasilnya di data validation.”

Slide 46/72 Embedding: dari indeks kata ke vektor

Ucapkan: ”Bagian terakhir yang dibutuhkan sebelum teks bisa masuk ke LSTM. Ikuti empat kotak di gambar. Kata bagus dicari di kamus dan ketemu di nomor 187. Nomor itu dipakai mengambil satu baris dari tabel embedding, 10.000 baris kali 64 kolom. Hasilnya vektor 64 angka, dan vektor itulah yang masuk ke model. Kenapa tidak nomor 187 saja yang dimasukkan? Karena indeks hanyalah nomor urut di kamus: kata bernomor 5.000 bukan dua kali lipat kata bernomor 2.500, jadi angkanya tidak boleh masuk langsung ke layer yang menghitung jumlah berbobot. Yang perlu dicatat dari keterangan di bawah gambar: isi tabel itu dilatih bersama model, bukan ditetapkan sebelumnya. Dan supaya satu batch bisa dihitung bersama, di nb04 semua review disamakan menjadi 200 langkah: yang pendek ditambah nol di depan, yang panjang dipotong. Kata embedding kembali besok di Modul 3 dan 4.”

Slide 47/72 Hasil nyata: SimpleRNN memprediksi nilai berikutnya

Ucapkan: ”Hasil sungguhan dari nb04, dan syaratnya disebut dulu supaya angkanya punya arti. SimpleRNN(32) membaca 20 langkah terakhir dan memprediksi satu nilai berikutnya. MAE-nya 0,0246 pada data test, satu run, seed 42, 20 epoch. Di gambar, kurva prediksi menempel rapat pada kurva sebenarnya. Sebelum senang dengan angka itu, dua catatan. Pertama, pembandingnya: baseline ulangi nilai terakhir yang dilihat, dan di nb04 baseline itu dihitung lebih dulu, sebelum modelnya, supaya angka model punya pembanding. Kebiasaan itu yang dibawa dari hari satu. Kedua, batasnya: pola berulang serapi gelombang sinus ini memang mudah diprediksi. Deret nyata seperti penjualan atau bacaan sensor punya tren dan gangguan, jadi MAE sekecil ini tidak berpindah begitu saja ke data lapangan.”

Slide 48/72 Kapan model urutan cocok, dan apa jebakannya

Ucapkan: "Kolom kiri, keadaan yang cocok: urutannya bermakna dan panjangnya puluhan sampai ratusan langkah; datanya deret waktu dari sensor; atau modelnya perlu kecil, misalnya untuk dipasang di perangkat edge, dan RNN memang jauh lebih ringan daripada transformer. Kolom kanan, keadaan yang kurang cocok: konteks jauh yang menentukan, dan untuk itu transformer di Modul 3 dan 4 lebih tepat; urutannya bisa diringkas menjadi feature tabular, misalnya rata-rata dan tren tujuh hari, sehingga model kemarin sudah cukup; atau barisnya sebenarnya tidak berurutan. Kotak jebakan: LSTM tidak otomatis lebih baik daripada SimpleRNN. Pada urutan pendek seperti data sinus nb04, keduanya bisa setara sementara LSTM memakai sekitar empat kali lebih banyak bobot. Yang menentukan pilihannya hasil di data validation, bukan reputasi namanya."

Slide 49/72 Ringkasan Act 4

Ucapkan: "Lima hal dari blok urutan. Satu, pada data berurutan posisi ikut membawa arti, jadi modelnya perlu membawa sesuatu dari langkah sebelumnya. Dua, RNN menyimpan hidden state, satu ringkasan pendek yang diperbarui tiap langkah dengan bobot yang sama di sepanjang urutan. Tiga, makin panjang urutannya, makin tipis sinyal perbaikan yang sampai ke langkah awal; itu vanishing gradient dalam bentuk waktu. Empat, LSTM menambahkan catatan dengan tiga gerbang terlatih, dan GRU melakukan hal serupa dengan dua gerbang serta bobot lebih sedikit. Lima, teks masuk lewat layer Embedding yang menukar indeks kata menjadi vektor, dan panjang urutannya disamakan lebih dulu."

Act 5 — Kapan GPU benar-benar diperlukan?

Slide 50/72 Act 5: Kapan GPU benar-benar diperlukan?

Ucapkan: "Blok kelima, dan berhenti soal arsitektur. Empat notebook tadi semuanya jalan lebih cepat di GPU, dan pertanyaannya kenapa, sampai batas mana, dan apa yang dilakukan setelah training selesai. Judul kecil di bawah menyebut jalurnya: dari perkalian matriks sampai TensorRT."

Slide 51/72 Training berisi banyak perkalian matriks besar

Ucapkan: "Sebelum bicara perangkat, kita lihat bentuk pekerjaannya. Satu batch masukan berukuran 64 kali 784: 64 gambar, tiap gambar 784 angka. Bobot satu layer Dense berukuran 784 kali 128. Hasil perkaliannya grid di kanan, 64 kali 128. Sekarang kalimat di kanan gambar, dan ini inti seluruh act ini: tiap sel keluaran dihitung dari barisnya sendiri dan kolomnya sendiri, jadi urutan pengerjaannya tidak penting. Empat sel hijau di grid itu bisa dihitung bersamaan tanpa saling menunggu. Dua butir berikutnya soal jumlahnya. Satu langkah training menghitung prediksi maju, lalu membagi kesalahan ke tiap bobot mundur, dan keduanya berisi perkalian matriks seperti ini. Benchmark nb05 melatih CNN pada 10.000 gambar CIFAR-10 dengan batch

64 selama beberapa epoch, jadi perkalian seperti ini terjadi ribuan kali.”

Slide 52/72 Intuisi: CPU sedikit tapi kuat, GPU banyak tapi sederhana

Ucapkan: ”Analogi dari nb05. Nilai ujian seribu siswa perlu dihitung. Pilihan pertama, satu guru yang sangat cepat dan bisa mengerjakan apa saja, termasuk menangani lembar jawaban yang aneh. Pilihan kedua, seribu kalkulator sederhana yang bekerja bersamaan, masing-masing hanya bisa menjumlahkan. Untuk pekerjaan seperti tadi, pilihan kedua yang menang, dan syaratnya sudah kita lihat di slide sebelumnya: pekerjaannya bisa dipecah menjadi bagian-bagian yang tidak saling menunggu. Kembali ke mekanismenya: CPU punya sedikit unit hitung yang kuat dan serba bisa, kotak besar di kiri; GPU punya banyak unit hitung sederhana yang bekerja serentak, petak-petak kecil di kanan. Analoginya berhenti di situ. Sekali pekerjaannya tidak bisa dipecah, seribu kalkulator itu hanya menunggu, dan itu isi slide berikutnya.”

Slide 53/72 Kapan GPU tidak memberi keuntungan

Ucapkan: ”Empat keadaan, dan halaman ini yang paling sering hilang dari materi tentang GPU. Modelnya kecil: beberapa layer Dense di atas data tabular selesai sebelum GPU sempat sibuk. Batchnya kecil: batch delapan memberi sedikit pekerjaan per langkah, jadi banyak unit hitung menganggur. Datanya bolak-balik: setiap batch harus dipindahkan ke memori GPU dulu, dan kalau hitungannya ringan, pemindahan itu yang mendominasi waktunya. Pemanggilan pertama mahal: kernel dan graph disiapkan saat dipakai pertama kali, jadi panggilan pertama jauh lebih lambat daripada panggilan berikutnya. Kotak jebakan menyambung ke butir keempat: mengukur tanpa warm-up membuat GPU tampak lambat. Di nb05, `fit` dan `predict` dipanggil sekali di kedua sisi sebelum stopwatch dinyalakan, supaya yang dibandingkan hitungannya, bukan persiapannya. Kalau hasil pengukuranmu menunjukkan CPU lebih cepat, itu temuan yang sah, bukan kesalahan.”

Slide 54/72 Mixed precision: hitung di float16, keluaran di float32

Ucapkan: ”Cara paling murah mempercepat training di GPU modern: tiga baris kode di atas. Baris pertama impor, baris kedua menyalakan policy mixed float enam belas untuk seluruh model, baris ketiga mengecualikan output layer. Alasannya di butir pertama dan kedua. Float enam belas memakai 16 bit per angka, float tiga dua memakai 32 bit, jadi angkanya lebih hemat memori dan lebih cepat dipindahkan, dan GPU NVIDIA modern punya Tensor Core yang memang dibuat untuk operasi float enam belas. Tetapi ada bagian yang rawan kalau jangkauan angkanya dipersempit: keluaran softmax dan penjumlahan gradient. Bagian itu dibiarkan di float tiga dua, dan karena itu output layernya diberi `dtype` sama dengan float tiga dua. Butir ketiga menjaga harapan tetap wajar: ini bukan tombol ajaib. Keuntungannya terasa pada model dan batch yang cukup besar, dan besarnya diukur di nb05, bukan diperkirakan. Catatan kecil di bawah: di nb05 policynya dikembalikan ke float tiga dua setelah percobaan supaya sel-sel berikutnya tidak ikut terpengaruh.”

Slide 55/72 Dari training ke inferensi: ONNX lalu TensorRT

Ucapkan: "Sampai sini semua soal training. Sekarang modelnya sudah jadi dan tinggal dipakai, dan pekerjaan itu punya alat sendiri. Ikuti rantai di gambar: model Keras hasil training diekspor menjadi SavedModel, dikonversi dengan `tf2onnx` menjadi ONNX, lalu dibangun dengan Polygraphy menjadi engine TensorRT FP16, dan engine itulah yang dipakai melayani inferensi. Kenapa lewat ONNX? Karena ONNX format perantara: sekali model ada di sana, banyak mesin inferensi bisa membacanya tanpa peduli framework asalnya. Sekarang syarat versinya, dan bagian ini penting supaya notebooknya tidak gagal di kelas. TF-TRT, integrasi bawaan TensorFlow, sudah dihapus sejak TensorFlow 2.18; versi 2.17 yang terakhir memuatnya, dan Colab sekarang memakai 2.18 ke atas. Karena itu jalurnya lewat ONNX. Lalu dua pin di baris `pip` itu: `tensorrt` dipin ke seri sepuluh titik x, karena TensorRT 11 menghapus cara mudah menyalakan FP16 saat engine dibangun; dan `tf2onnx` minimal 1.17 supaya cocok dengan NumPy 2."

Slide 56/72 Benchmark: angkanya diukur di kelas, bukan dijanjikan di slide

Ucapkan: "Panel bergaris putus-putus di atas ini memang belum berisi angka, dan itu disengaja. Angka kecepatan GPU hanya berarti bersama syarat pengukurannya, dan syaratnya baru ada setelah nb05 dijalankan di runtime hari ini. Jadi yang kita baca sekarang syaratnya dulu. Yang diukur nb05 ada empat: perkalian matriks berukuran 500 sampai 8.000; training CNN tiga epoch pada 10.000 gambar CIFAR-10; inferensi 1.000 gambar; dan TensorRT FP16 dibandingkan TensorFlow FP32 untuk model yang sama. Pembandingnya CPU Colab lawan satu GPU T4 pada runtime yang sama, dan tiap sisi dipanggil sekali sebagai warm-up sebelum diukur. Nanti setelah nb05 selesai, figur ini terisi dari hasil run kita sendiri, dan angkanya milik sesi masing-masing. Tanya ke kelas setelah nb05 jalan: dari empat tugas itu, mana yang selisihnya paling kecil, dan apa yang membedakannya dari tiga yang lain?"

Slide 57/72 Kapan GPU sepadan, dan apa jebakannya

Ucapkan: "Kolom kiri, keadaan yang sepadan: datanya gambar atau urutan panjang dan modelnya berlapis; batchnya cukup besar dan epochnya banyak; banyak percobaan, artinya arsitekturnya diganti-ganti dan trainingnya diulang; atau inferensinya melayani banyak permintaan sekaligus. Kolom kanan, keadaan yang kurang sepadan: data tabular kecil seperti hari satu; satu prediksi sesekali di server biasa; hitungannya ringan sehingga waktunya habis untuk memindahkan data; dan yang terakhir ini paling sering terjadi, modelnya belum benar sehingga masalahnya bukan kecepatan. Kotak jebakan menegaskan itu: GPU mempercepat hitungan, bukan memperbaiki model. Kalau lossmu tidak turun, yang perlu diperiksa data, learning rate, dan arsitekturnya. GPU hanya membuat percobaan yang sama selesai lebih cepat, termasuk percobaan yang salah."

Slide 58/72 Ringkasan Act 5

Ucapkan: "Lima hal dari blok GPU. Satu, training deep learning berisi perkalian matriks besar yang bisa dipecah, dan itulah bentuk pekerjaan yang cocok untuk GPU. Dua, CPU punya sedikit unit hitung yang kuat, GPU punya banyak unit sederhana yang bekerja serentak. Tiga, keuntungannya hilang kalau modelnya kecil, batchnya kecil, atau waktunya habis memindahkan data, dan warm-up harus dikeluarkan dari pengukuran. Empat, mixed precision menghitung di float enam belas dan menyimpan bagian yang rawan di float tiga dua, dengan output layer diberi dtype float tiga dua. Lima, untuk inferensi model diekspor lewat ONNX lalu dibangun menjadi engine TensorRT, karena jalur bawaan TF-TRT sudah tidak ada di TensorFlow 2.18 ke atas. Jalur ini muncul lagi di Modul 6."

Act 6 — Bagaimana model menghasilkan data baru?

Slide 59/72 Act 6: Bagaimana model menghasilkan data baru?

Ucapkan: "Blok terakhir yang berisi materi baru, dan arahnya berbalik. Lima blok tadi semuanya tentang memberi label atau angka pada data yang masuk. Pertanyaannya sekarang: bagaimana model menghasilkan datanya sendiri. Tiga jawaban, seperti judul kecilnya: memampatkan, berlomba, lalu membalik noise."

Slide 60/72 Dua arah: menebak label, atau menghasilkan data

Ucapkan: "Dua baris, dua arah kerja. Baris atas diskriminatif, dan itu nb01 sampai nb04: gambar masuk, model memberi label kaus. Baris bawah generatif, dan itu nb06: noise atau teks masuk, keluar gambar baru. Bedanya bukan tingkat kesulitan, tetapi apa yang dilatih. Model diskriminatif dilatih memisahkan; model generatif dilatih mengenali seperti apa data yang wajar, lalu mengambil contoh baru dari situ. Butir terakhir yang jadi pola bersama ketiga metode di act ini: bahan dasarnya sesuatu yang sederhana, biasanya noise acak, yang diubah bertahap menjadi data yang rumit. Satu akibat praktis yang perlu disebut sekarang dan dibahas lagi di Slide 67: karena tidak ada satu jawaban benar, akurasi tidak bisa dipakai menilai model generatif."

Slide 61/72 Intuisi: Autoencoder: peras dulu, lalu bangun ulang

Ucapkan: "Bentuk jam pasir di gambar itu seluruh idenya. Gambar 28 kali 28, jadi 784 angka, masuk ke encoder yang menyempit sampai tinggal 32 angka di tengah. Lalu decoder melebar lagi dan membangun ulang gambar 28 kali 28. Lossnya, tulisan di bawah, selisih antara gambar yang masuk dan gambar yang keluar; jadi model ini dilatih tanpa label sama sekali, targetnya masukannya sendiri. Sekarang mekanisme yang membuatnya menarik. Karena bagian tengahnya hanya 32 angka, model tidak mungkin menyalin; dia harus memilih apa yang dipertahankan. Kalau gambarnya bisa dibangun ulang dari 32 angka, maka 32 angka itu memuat isi pentingnya. Dan begitu decodernya bisa bekerja, dia bisa dipakai sendiri tanpa

encoder: masukkan 32 angka, keluar satu gambar. Di situlah autoencoder mulai menjadi generator.”

Slide 62/72 Latent space: alamat 32 angka untuk tiap gambar

Ucapkan: ”Ini hasil sungguhan, dan syaratnya di keterangan bawah: autoencoder Dense pada MNIST, latent 32 angka, 10 epoch, seed 42. Cara gambarnya dibuat: alamat 32 angka untuk satu digit tujuh diambil, alamat untuk satu digit dua diambil, lalu jalan lurus dari alamat pertama ke alamat kedua dilewati sedikit demi sedikit, dan tiap titik di jalan itu di-decode menjadi gambar. Hasilnya baris gambar dari kiri ke kanan. Yang perlu dilihat: bentuk di antaranya tetap masuk akal, bukan tumpukan dua gambar yang saling menembus. Artinya latent space menyimpan struktur, bukan hanya memampatkan gambar satu per satu. Satu batas yang jujur: titik acak yang jauh dari data justru memberi gambar kabur, karena autoencoder biasa tidak pernah diminta merapikan seluruh ruangnya, hanya bagian yang dilewati data train. VAE menambahkan aturan itu, dan itu baris kedua di tabel dua slide lagi.”

Slide 63/72 Intuisi: GAN: pemalsu lukisan melawan kurator galeri

Ucapkan: ”Dua pemain, dan keduanya berlatih dari pertandingan yang sama. Pemain pertama pemalsu lukisan: dari noise acak, dia membuat gambar. Pemain kedua kurator galeri: dia diberi campuran gambar palsu dan gambar asli dari MNIST, dan tugasnya menjawab satu pertanyaan, asli atau palsu. Sekarang mekanismenya, dan itu panah balik di bawah gambar. Jawaban kurator dipakai dua kali: menjadi loss bagi kurator, salah tebakan berapa banyak, dan menjadi loss bagi pemalsu, seberapa sering tiruannya tertangkap. Karena itu keduanya dilatih bersamaan, dan setiap kali kurator makin jeli, pemalsu terpaksa membuat tiruan yang lebih meyakinkan, lalu sebaliknya. Satu hal yang praktis: yang kita simpan di akhir hanya generatornya. Kuratornya alat latihan, dan setelah training selesai dia tidak dipakai lagi.”

Slide 64/72 Digit yang lahir dari noise, epoch demi epoch

Ucapkan: ”Hasil nyata: DCGAN pada MNIST, 15 epoch, seed 42. Cara membaca grid ini yang penting. Tiap baris memakai noise masukan yang sama, jadi yang berubah dari kolom ke kolom hanya generatornya. Kolom epoch satu masih berbintik, hampir tidak berbentuk. Kolom epoch lima mulai punya goresan. Kolom epoch 15 sebagian sudah berbentuk digit yang bisa dikenali, dan tidak satu pun dari gambar itu ada di dataset; semuanya keluar dari noise. Kotak jebakan menyebut kegagalan yang paling khas dari GAN: mode collapse. Generator bisa menemukan satu atau dua bentuk yang lolos terus dari kurator, lalu berhenti belajar variasi lain. Gejalanya keluaran yang mirip satu sama lain, jadi yang diperiksa keragamannya, bukan hanya kerapiannya. Satu grid yang isinya sepuluh angka delapan yang bagus adalah GAN yang gagal.”

Slide 65/72 Diffusion: noise ditambahkan bertahap, lalu dibalik

Ucapkan: "Pendekatan ketiga, dan yang dipakai model gambar yang mungkin sudah kamu coba. Empat kotak di gambar, dari kiri ke kanan: digit tujuh yang jelas, lalu makin berbintik, sampai kotak terakhir yang isinya noise murni. Panah hijau di atas arah maju, menambahkan noise sedikit demi sedikit, dan arah itu tidak perlu dilatih; menambah noise bisa dihitung langsung. Panah oranye di bawah arah balik, membersihkan selangkah demi selangkah, dan inilah satu-satunya bagian yang dipelajari model: dari gambar yang agak berbintik, tebak versi yang sedikit lebih bersih. Sekarang biaya jujurnya. Melatih model diffusion dari nol butuh data dan waktu jauh di luar satu sesi kelas. Inferensinya juga bertahap: satu gambar butuh beberapa sampai puluhan langkah pembersihan, bukan satu lompatan seperti GAN. Karena itu di nb06 kita memakai model pretrained SD-Turbo dengan empat langkah, dan menyimpan gambar di tiap langkah supaya prosesnya terlihat."

Slide 66/72 Peta keluarga generatif

Ucapkan: "Satu tabel untuk merapikan lima nama yang beredar. Autoencoder: peras lalu bangun ulang, decodernya jadi generator, dan dipakai untuk kompresi dan denoising. VAE: latent spacenya diatur supaya titik acak tetap memberi hasil wajar, dan itu jawaban atas batas yang kita lihat di Slide 62. GAN: generator berlomba melawan discriminator, contohnya StyleGAN. Diffusion: belajar membersihkan noise bertahap, contohnya Stable Diffusion. Baris terakhir, Transformer: menebak potongan berikutnya, berulang-ulang. Kotak di bawah menyebut kenapa baris terakhir itu ada di sini. Besok kita pakai baris itu sepanjang hari, dan yang berubah bahan keluarannya: bukan pixel lagi, tetapi token, satu potongan teks demi satu potongan teks. Idenya tetap sama, menghasilkan sesuatu yang wajar menurut data yang pernah dilihat."

Slide 67/72 Kapan model generatif cocok, dan apa jebakannya

Ucapkan: "Kolom kiri, keadaan yang cocok: butuh contoh baru, misalnya menambah variasi data atau mengisi bagian yang hilang; butuh ringkasan padat, jadi kompresi, denoising, dan deteksi anomali dari galat rekonstruksi; atau hasilnya dinilai orang, bukan dicocokkan dengan satu jawaban. Kolom kanan, keadaan yang kurang cocok: pertanyaannya klasifikasi atau prediksi satu angka; jawabannya harus bisa diperiksa benar atau salah; atau data trainnya sedikit dan sempit, sehingga yang dihasilkan hanya mengulang yang itu-itu saja. Kotak jebakan ini yang paling penting dibawa ke besok. Model generatif menghasilkan yang wajar menurut data trainnya, belum tentu yang benar, dan akurasi tidak bisa dipakai menilainya. Di Modul 5 kita lihat satu cara mengikat keluaran ke sumber yang bisa ditunjuk, supaya klaimnya bisa diperiksa."

Slide 68/72 Ringkasan Act 6

Ucapkan: "Lima hal dari blok generatif. Satu, model diskriminatif memetakan data ke label, sementara model generatif belajar seperti apa data yang wajar lalu menghasilkan contoh baru. Dua, autoencoder memampatkan gambar menjadi beberapa angka dan membangunnya ulang, dan

decodernya bisa dipakai sendiri sebagai generator. Tiga, latent space menyimpan struktur, terlihat dari interpolasi yang mulus, sementara titik acak di daerah kosong masih memberi hasil kabur. Empat, GAN melatih generator dan discriminator sebagai lawan tanding; hasilnya bisa bagus, tetapi rewel dan rentan mode collapse. Lima, diffusion membalik penambahan noise selangkah demi selangkah: mahal saat training, dan bertahap saat menghasilkan gambar.”

Penutup

Slide 69/72 Enam notebook hari ini

Ucapkan: ”Seluruh daftarnya, dan kolom data yang perlu diperhatikan. nb01, neural network pertama dengan Keras, Fashion-MNIST. nb02, fungsi aktivasi dan optimizer, data yang sama supaya perbandingannya adil. nb03, CNN dan transfer learning, CIFAR-10. nb04, RNN, LSTM, dan embedding, deret sinus dan review IMDB. nb05, GPU, mixed precision, ONNX, TensorRT, CIFAR-10 lagi. nb06, autoencoder, GAN, dan diffusion, MNIST dan SD-Turbo yang sudah pretrained. Dua kebiasaan yang berlaku di keenamnya, ada di catatan bawah: `set_seed` dipasang supaya hasilnya bisa diulang, dan setiap model dibandingkan dengan baseline lebih dulu. Semuanya jalan di Colab dengan runtime T4, dan tiap notebook memuat datanya sendiri, jadi kalau satu macet, berikutnya tetap bisa dibuka.”

Slide 70/72 Apa yang menyusul sesudah hari 2

Ucapkan: ”Empat modul di depan, dan yang perlu ditekankan bekal hari ini dipakai terus. Modul 3, NLP: teks menjadi angka, token dan vektor kata, dan layer Embedding tadi pintu masuknya. Modul 4, LLM: transformer dan model bahasa. Modul 5, RAG: menjawab dengan sumber yang bisa ditunjuk, dan itu jawaban atas jebakan di Slide 67. Modul 6, ekosistem NVIDIA: optimasi, deploy, dan sisi tanggung jawab, dan jalur ONNX serta TensorRT hari ini muncul lagi di sana, kali ini untuk model bahasa. Tiga butir di bawah menyebut yang dibawa: loss, gradient descent, overfitting, embedding, dan cara memakai GPU. Model urutan hari ini juga pintu masuk ke transformer, jadi kalau Act 4 terasa berat, bagian itu akan terbayar besok.”

Slide 71/72 Latihan mandiri sebelum hari 3

Ucapkan: ”Satu permintaan sebelum ditutup. Tiap notebook punya blok Latihan di bagian akhir, dengan satu sel kosong untuk kodemu. Kerjakan minimal satu sebelum hari 3, dan pilih yang paling membuat penasaran, bukan yang paling mudah. Dua contoh yang saya sarankan. Di nb05, kecilkan jumlah gambar inferensi sampai keuntungan GPU hilang; angka batas itu jauh lebih berguna diingat daripada satu angka speedup. Di nb06, ubah `LATENT_DIM` autoencoder menjadi 8 atau 64, lalu lihat rekonstruksinya. Dan satu hal teknis yang menentukan latihannya berguna atau tidak: simpan hasil sebelum dan sesudah perubahan. Kotak di bawah itu kebiasaan yang saya harap terbawa dari hari ini. Sebelum menyimpulkan satu perubahan menolong, catat syaratnya: data mana, berapa epoch, seed berapa, dan baseline apa.”

Slide 72/72 Terima kasih

Ucapkan: "Terima kasih sudah bertahan satu hari penuh. Kalau hanya satu hal yang terbawa dari hari ini, saya berharap yang terbawa jalur pendek ini: hitung lossnya, cari kemiringannya, ambil satu langkah kecil, ulangi. Enam notebook hari ini hanya variasi dari itu. Sampai besok di Modul 3, NLP."